

M907 – Sistemas Distribuídos

Prof. Nabor das Chagas Mendonça

RELATÓRIO TÉCNICO – ETAPA 2

Observação da Aplicação e Testes de Desempenho

Sock Shop – Microservices Demo Application

Izabella Maria Pinheiro Amorim – Matrícula: 2527553

Leticia Vasconcelos Machado – Matrícula: 2418950

Leonardo de Freitas Rabelo – Matrícula: 2651489

Renan Medeiros Aguiar – Matrícula: 2527541

Fortaleza, maio de 2026

1. Introdução

Esta segunda etapa do relatório técnico dá continuidade ao trabalho iniciado na Etapa 1, na qual foi realizada a implantação da aplicação Sock Shop em ambiente Kubernetes local utilizando o Docker Desktop em um MacBook com processador Apple M2. Tendo a aplicação operacional, o objetivo desta etapa é observar seu comportamento em condições normais de operação e submetê-la a testes de desempenho sob carga controlada.

Para isso, foi utilizada a ferramenta Locust, geradora de carga baseada em Python, com scripts que simulam o comportamento de usuários reais navegando pela loja virtual. Foram realizados dois experimentos principais: um com 100 usuários simultâneos e outro com 1.000 usuários, permitindo observar a degradação progressiva do desempenho da aplicação conforme a carga aumenta.

O relatório documenta a metodologia adotada, os parâmetros configurados, os resultados obtidos e uma análise comparativa entre os cenários testados.

2. Aplicação Escolhida

A aplicação utilizada nesta atividade é o Sock Shop, desenvolvido pela Weaveworks como demonstração de referência de arquitetura de microsserviços. Trata-se de um sistema de e-commerce simulado, composto por múltiplos serviços independentes que interagem via HTTP (REST) e mensageria assíncrona.

Repositório Principal

<https://github.com/microservices-demo/microservices-demo>

Arquitetura Resumida

O Sock Shop é composto pelos seguintes microsserviços principais:

- front-end – Interface web da loja (Node.js), ponto de entrada dos usuários
- catalogue / catalogue-db – Catálogo de produtos com banco MongoDB
- carts / carts-db – Gerenciamento de carrinho de compras
- orders / orders-db – Processamento de pedidos
- payment – Serviço de pagamento
- shipping – Serviço de entrega
- user / user-db – Gerenciamento de usuários e autenticação
- queue-master / rabbitmq – Fila de mensagens para processamento assíncrono
- session-db – Armazenamento de sessões (Redis)

Justificativa da Escolha

O Sock Shop foi selecionado por sua alta adequação para uso em laboratório, conforme identificado na Etapa 1. A aplicação possui suporte nativo a Kubernetes, manifesto de implantação completo, diversidade tecnológica entre os serviços e um fluxo funcional rico para geração de carga representativa, além de ser amplamente utilizada em contextos acadêmicos.

3. Ambiente Experimental

Os testes foram realizados no seguinte ambiente:

Componente	Descrição
Hardware	MacBook Pro com Apple M2 (ARM64)
Sistema Operacional	macOS (versão compatível com Docker Desktop)
Orquestrador	Kubernetes via Docker Desktop
Distribuição Kubernetes	Docker Desktop Kubernetes (cluster local)
Ferramenta de carga	Locust 2.x (execução local via pip)
Acesso à aplicação	http://localhost (front-end na porta 80)
Repositório	microservices-demo (GitHub, Weaveworks)

Observações sobre a Implantação

Devido à arquitetura ARM64 do processador M2, algumas imagens Docker originais do Sock Shop não possuem suporte nativo a essa plataforma. Durante a implantação, os seguintes ajustes foram necessários:

- Substituição das imagens de carts-db e orders-db por mongo:4.4, compatível com ARM64
- Substituição da imagem do rabbitmq por rabbitmq:3.8-management-alpine
- Substituição da imagem do session-db por redis:alpine

Essas adaptações foram realizadas via kubectl patch e reaplicadas com manifestos personalizados, mantendo a compatibilidade funcional dos serviços. Ao momento dos testes, os seguintes serviços estavam operacionais:

- front-end, catalogue, catalogue-db, orders, payment, shipping, user, user-db, queue-master – status Running
- carts-db, orders-db, rabbitmq, session-db – em processo de estabilização (problemas de pull de imagem em redes instáveis)

4. Cenário Funcional e Script de Carga

4.1 Cenário Definido

O cenário funcional escolhido simula um usuário típico navegando pela loja virtual, com as seguintes ações ponderadas por frequência de uso:

Ação	Endpoint	Peso	% Aproximada
Acessar página inicial	GET /	10	~47%
Navegar pelo catálogo	GET /catalogue	5	~24%
Ver produto específico	GET /catalogue/{id}	3	~14%
Acessar carrinho	GET /cart	2	~10%
Fazer login	GET /login	1	~5%

Os pesos refletem o comportamento real de usuários em e-commerce: a maioria acessa a página inicial e navega pelo catálogo, enquanto apenas uma fração menor realiza login ou acessa o carrinho. O intervalo de espera entre ações foi configurado entre 1 e 3 segundos, simulando o tempo de leitura e decisão do usuário.

4.2 Ferramenta Utilizada

A ferramenta de geração de carga utilizada foi o Locust, instalado localmente via pip3. O Locust foi escolhido por ser open source, baseado em Python, possuir interface web intuitiva para configuração e monitoramento em tempo real, e gerar relatórios detalhados em HTML e CSV.

4.3 Script de Carga (locustfile.py)

O script utilizado nos testes foi:

```
from locust import HttpUser, task, between
class SockShopUser(HttpUser):
    wait_time = between(1, 3)
    @task(10)
    def index(self): self.client.get("/")
    @task(5)
    def catalogue(self): self.client.get("/catalogue")
    @task(3)
    def add_to_cart(self): self.client.get("/catalogue/3395a43e-...")
    @task(2)
    def cart(self): self.client.get("/cart")
    @task(1)
    def login(self): self.client.get("/login", auth=("user", "password"))
```

5. Metodologia dos Testes

Foram executados dois experimentos distintos, variando o número de usuários simultâneos para avaliar o comportamento da aplicação sob diferentes níveis de carga. Em ambos os casos, o host alvo foi `http://localhost`, correspondente ao serviço front-end do Sock Shop.

Parâmetro	Experimento 1	Experimento 2
Usuários simultâneos	100	1.000
Ramp up (usuários/s)	10	50
Host alvo	<code>http://localhost</code>	<code>http://localhost</code>
Intervalo entre ações	1–3 segundos	1–3 segundos
Ferramenta	Locust	Locust

O ramp up diferenciado entre os experimentos foi intencional: no teste com 100 usuários, utilizou-se 10 usuários por segundo para uma rampa de 10 segundos até atingir a carga total; no teste com 1.000 usuários, 50 usuários por segundo resultando em 20 segundos de rampa, estressando a aplicação de forma mais agressiva para observar degradação gradual nos gráficos.

6. Resultados Obtidos

6.1 Experimento 1 – 100 Usuários Simultâneos

No primeiro experimento, a aplicação apresentou desempenho satisfatório, com tempos de resposta baixos e zero falhas em todos os endpoints.

Endpoint	Requisições	Falhas	Média (ms)	Mediana (ms)	Máx (ms)	p95 (ms)	p99 (ms)	RPS
GET /	11.687	0	6,6	5	180	13	46	23,49
GET /cart	2.275	0	47,9	50	550	65	160	4,57
GET /catalogue	5.799	0	10,1	8	184	18	50	11,66
GET /catalogue/{id}	3.583	0	41,4	48	164	56	71	7,20
GET /login	1.172	0	56,3	57	482	83	160	2,36

Com 100 usuários, a aplicação operou dentro de parâmetros excelentes. O endpoint mais acessado (página inicial) respondeu com mediana de apenas 5ms e p95 de 13ms, indicando alta disponibilidade e baixa latência. Não foram registradas

falhas em nenhum dos cinco endpoints monitorados. A taxa total de requisições atingiu aproximadamente 48,3 RPS (soma dos endpoints).

6.2 Experimento 2 – 1.000 Usuários Simultâneos

No segundo experimento, com dez vezes mais usuários, foram observadas degradações significativas no desempenho, embora o número absoluto de falhas tenha permanecido baixo em relação ao volume total de requisições.

Endpoint	Requisições	Falhas	Média (ms)	Mediana (ms)	Máx (ms)	p95 (ms)	p99 (ms)	RPS
GET /	34.894	0	1.191	17	41.961	770	35.000	109,19
GET /cart	6.936	3	2.061	68	47.503	21.000	44.000	21,70
GET /catalogue	17.767	6	1.906	30	43.744	19.000	42.000	55,60
GET /catalogue/{id}	10.687	2	1.853	66	43.689	17.000	42.000	33,44
GET /login	3.477	1	2.378	83	47.513	28.000	45.000	10,88

Com 1.000 usuários, a aplicação continuou respondendo, porém com degradação substancial nos tempos de resposta médios e nos percentis superiores. O tempo médio da página inicial passou de 6,6ms para 1.191ms (aumento de ~180x), e o p95 saltou de 13ms para 770ms. A taxa total de requisições cresceu para aproximadamente 230 RPS, demonstrando que a aplicação ainda processava a carga, porém com latência elevada nas caudas de distribuição.

6.3 Comparação entre os Experimentos

Métrica	100 usuários	1.000 usuários	Variação
Total de requisições (GET /)	11.687	34.894	+199%
Tempo médio GET /	6,6 ms	1.191 ms	+18.048%
Mediana GET /	5 ms	17 ms	+240%
p95 GET /	13 ms	770 ms	+5.823%
p99 GET /	46 ms	35.000 ms	+76.000%
Máximo GET /	180 ms	41.961 ms	+23.201%
RPS total (GET /)	23,49	109,19	+365%
Falhas totais	0	12	—

A tabela evidencia que, embora a aplicação mantenha alta disponibilidade (taxa de falha inferior a 0,02% com 1.000 usuários), a latência cresce de forma não linear. A mediana permanece relativamente estável (5ms vs 17ms), sugerindo que a maioria das requisições é atendida rapidamente, mas os percentis superiores (p95, p99) revelam uma cauda longa, indicando que uma parcela das requisições sofre filas ou timeouts intermitentes.

7. Análise e Discussão dos Resultados

7.1 Comportamento sob Carga Médica (100 usuários)

Com 100 usuários simultâneos, o Sock Shop demonstrou desempenho altamente satisfatório. A ausência total de falhas e os tempos de resposta na casa de milissegundos indicam que a aplicação opera confortavelmente dentro de sua capacidade nesse nível de carga. O throughput estabilizado em ~48 RPS confirma que os serviços principais (front-end, catalogue, orders) responderam de forma consistente.

7.2 Comportamento sob Alta Carga (1.000 usuários)

Com 1.000 usuários, observou-se um padrão clássico de degradação: enquanto a mediana de tempo de resposta permaneceu relativamente baixa (17ms para a página inicial), os percentis superiores apresentaram crescimento expressivo. Esse fenômeno é típico de situações em que o sistema processa a maioria das requisições rapidamente, mas sofre com congestionamento pontual em momentos de pico.

Contribuíram para essa degradação fatores inerentes ao ambiente de teste: o cluster Kubernetes local compartilha os recursos de CPU e memória do MacBook M2 com o sistema operacional e outros processos. Em um ambiente de produção com múltiplos nós dedicados, o comportamento tenderia a ser significativamente mais estável.

7.3 Análise por Endpoint

O endpoint /login apresentou os maiores tempos médios de resposta (56ms com 100 usuários e 2.378ms com 1.000 usuários), o que é esperado dado que envolve autenticação e consulta ao user-db. O endpoint /cart também apresentou latência mais elevada em ambos os cenários, possivelmente impactado pela instabilidade do carts-db (em fase de estabilização durante os testes).

O endpoint / (página inicial) manteve a melhor relação custo-benefício, respondendo com mediana de 5ms mesmo sob alta carga, evidenciando que o serviço front-end lida bem com requisições de conteúdo estático.

7.4 Limitações do Ambiente

As principais limitações identificadas durante os experimentos foram:

- Recursos compartilhados: o cluster Kubernetes local utilizou recursos de CPU e memória do notebook em paralelo com outros processos do sistema
- Instabilidade de rede: problemas de conectividade durante o pull de imagens causaram delays na inicialização de alguns serviços (carts-db, orders-db, rabbitmq, session-db)
- Incompatibilidade de arquitetura: imagens Docker sem suporte ARM64 nativo exigiram substituições e ajustes nos manifestos de implantação
- Ausência de ferramenta de observabilidade integrada: não foi possível coletar métricas internas dos pods (CPU, memória por serviço) durante os testes

8. Conclusão da Etapa 2

Os experimentos realizados nesta etapa demonstraram que o Sock Shop, mesmo em um ambiente local com recursos limitados, apresenta comportamento robusto sob cargas moderadas e gerenciável sob alta carga. Com 100 usuários, a aplicação operou sem falhas e com latência excelente. Com 1.000 usuários, manteve alta disponibilidade (falha < 0,02%), porém com degradação significativa nos percentis superiores, sugerindo gargalos de recursos computacionais no ambiente local.

Esses resultados fornecem uma base quantitativa importante para a Etapa 3, na qual serão explorados testes de resiliência, simulação de falhas e mecanismos de escalonamento automático (HPA), a fim de avaliar a capacidade da aplicação de adaptar-se dinamicamente a condições adversas.

O uso do Locust mostrou-se altamente adequado para este tipo de avaliação, oferecendo flexibilidade na definição de cenários, visualização em tempo real e relatórios detalhados que facilitam a análise comparativa entre experimentos.

Referências

WEAVEWORKS. Sock Shop Microservices Demo Application. Disponível em: <https://github.com/microservices-demo/microservices-demo>. Acesso em: mai. 2026.

LOCUST. Locust Documentation. Disponível em: <https://docs.locust.io>. Acesso em: mai. 2026.

KUBERNETES. Kubernetes Documentation. Disponível em: <https://kubernetes.io/docs>. Acesso em: mai. 2026.

NEWMAN, Sam. Building Microservices: Designing Fine-Grained Systems. 2. ed. Sebastopol: O'Reilly Media, 2021.